DIM0501 — Boas Práticas de Programação

Relatório do Projeto da Unidade 1 — AgendaFácil

Modelo de entrega — serve para Python, Java e C++

Prof. Marcus Terra · DIMAp/UFRN · Semestre 2026.1 · Entrega: 25/10/2026

Integrantes (nome e matrícula)	Linguagem
Jordson Albino Galvão Tavares - 20250023566 Rafaela Thais de Abreu Lúcio Maia - 20250025927	Python
Repositório ou arquivo entregue	
https://github.com/rafaalathais/projeto_agenda_bpp	

> **Como usar este modelo:** preencha as seis seções abaixo, apague os textos em itálico e entregue em PDF ou Markdown. O relatório inteiro deve caber em até quatro páginas — é um documento de decisões, não uma redação.

1. Diagnóstico do código-base

#	Categoria	Anomalia e onde está	Por que é um problema
1	Manutenibilidade	Nomes de variáveis pouco descritivas em add(p, pr, d, h, dur) e rem(p, d, h, nd, nh)	Dificulta a compreensão do que cada variável representa. Necessitando a leitura linha por linha do código.
2	Organização e estética	Cabeçalho contendo histórico de alteração de arquivo em # Alterado por: Joao - 15/05/2025 e # Alterado por: Marina - 09/01/2026	Polui o código com informações que deveriam ser gerenciadas no controle de versões (git).
3	Programação defensiva e segurança	Falta de validação para a variável data (d), permitindo qualquer string em add("Ana", "Dr. Silva", "2026-10-05", 9, 30)	Permite a inserção de dados inconsistentes, como datas que já passaram ou muito distantes, além de muitas formas de formatação para a data.
4	Desempenho e recursos	A função rem possui um loop de verificação aninhado percorrendo a lista ag duas vezes	Gera uma complexidade O(n^2), tornando a execução lenta caso a agenda tenha muitos registros.

5	Adequação e lógica	O comentário da função lst diz: “# lista os atendimentos do mes” mas ele faz um filtro por dia em if a[2] == d:	O comentário engana o leitor sobre a real funcionalidade da função, o que pode levar a confusões.
6	Manutenibilidade	As funções add e remp possuem mais de uma responsabilidade, elas realizam as verificações, agendam/remarcam e imprimem	Impacta na manutenção, pois em caso de alteração da lógica de imprimir seria preciso mexer em mais de um local do código.
7	Organização e estética	Comentários de código morto em # def desmarcar(p,d,h) # ag.remove(...)	Trechos de código comentados que nunca são executados, poluem o código e podem confundir a equipe.
8	Organização e estética	Comentários de código sujo em # adiciona na lista e # remarca o atendimento	Explicações de código confuso sem refatoração contribuem para a baixa manutenção.
9	Manutenibilidade	Números mágicos 8 e 18 soltos na função add()	Prejudica a legibilidade, pois esses elementos não são especificados causando confusão ao ler o código.
10	Programação defensiva e segurança	Os erros da função add() são silenciados no bloco try:.. execpt : pass	As exceções são ignoradas escondendo falhas no código e aumentando os riscos de segurança.

2. O que foi resolvido e o que ficou como dívida

#	Situação	Como foi resolvida, ou por que ficou
1	Resolvida	Nomes de variáveis e funções foram substituídos por nomenclaturas mais legíveis.
2	Resolvida	Os comentários foram removidos e foi integrado ao projeto o sistema de controle de versão git.
3	Resolvida	Uso de cláusula de guarda que levanta erro caso o dado seja inconsistente.

4	Resolvida	Uso de dataclass dispensando o acesso dos valores por meio de índice e loops aninhados.
5	Resolvida	Trecho removido
6	Resolvida	Criação de novas funções como validar_atendimento() e verificar_conflito_horario() diminuindo as responsabilidades das funções adicionar e remarcar, além da impressão para o usuário ocorrer na main.
7	Resolvida	Trechos removidos
8	Resolvida	Removidos, pois as docstrings possuem informação suficiente sobre o funcionamento da função e após a refatoração o código ficou autoexplicativo.
9	Resolvida	Os números soltos viraram constantes nomeadas.
10	Resolvida	Agora o except captura o erro e não silencia a falha.

3. Decisões de projeto

Decisão tomada	Alternativa descartada	Por que decidimos assim
Separar as responsabilidades das funções	Validar tudo dentro de adicionar_atendimento e remarcar_atendimento	Aplica o principio da responsabilidade única
Lançar exceções	Lançar valor padrão	Evita falhas silenciosas
Uso de dataclass	Acesso direto por índice	Proporciona tipagem explicita e acesso a atributos nomeados
Centralização da verificação de conflitos	Manter a verificação por meio de ciclos aninhados	Elimina a redundancia e inconsistencia

4. Mudanças de comportamento

Entrada ou situação	O que acontecia antes	O que acontece agora
Data fora do formato AAAA-MM-DD	Era inserida diretamente sem qualquer verificação	Válida via datetime.strptime e lança ValueError sobre qualquer formatação inválida
Horários marcados antes das 8h ou depois das 17h	O sistema ignorava em silêncio	Interrompe a execução lançando ValueError

Duração de atendimento fora do intervalo 15-120 minutos e diferente de um múltiplo de 15	Aceitava qualquer valor maior que 0	Lança ValueError indicando que a duração tem que está entre 15 e 120 minutos e ser múltipla de 15
Duração de atendimentos que excedem o horário final de expediente	O sistema não calculava o término da consulta e permitia agendar consultas fora do expediente	A função validar_atendimneto calcula o término e lança um ValueError impedindo o agendamento fora do expediente
Dados para remarcação de um atendimento que conflitavam com atendimentos já agendados	O sistema imprimia “Horario oculpado” sem interromper o erro	Lança um ValueError interrompendo o fluxo e tratando a violação de forma defensiva

5. Como executar e gerar a documentação

Item	Resposta
Comando para compilar e executar	py src/agenda_base.py
Comando para gerar a documentação de API	pdoc src/agenda_base.py -o docs/
Onde a documentação gerada ficou no repositório	Na pasta docs/ na raiz do repositório
Saída do programa de demonstração (cole aqui)	Atendimento marcado para Ana Horario ocupado Atendimento marcado para Carla Remarcado 14 Carla Dra. Souza MARCADO

6. Uso de ferramentas de IA

Pergunta	Resposta
Foram usadas ferramentas de IA neste trabalho?	( ) Não (x) Sim
Em quais etapas, e para quê?	Uso das bibliotecas internas datetime e dataclass para verificação do formato de data AAAA-MM-DD e uso de uma estrutura de dados com campos nomeados.

Conferência antes de enviar

- ☐ O programa de demonstração compila e roda, e imprime a mesma saída do código-base.
- ☐ Toda anomalia da seção 1 aparece na seção 2, resolvida ou declarada como dívida.
- ☐ O README permite que outra pessoa execute o projeto sem perguntar nada.
- ☐ Os textos em itálico deste modelo foram apagados.